



UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA
FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA
INGENIERÍA EN SOFTWARE Y TECNOLOGÍAS EMERGENTES

Gestión de la innovación (381)

Práctica 5

Análisis del impacto de la evolución tecnológica en la ingeniería de software

Ochoa Hernández Diego Alejandro | 1298530

Fecha de realización: 12 de abril de 2026

Fecha de entrega: 14 de abril de 2026

Resumen

El presente reporte técnico analiza de manera exhaustiva la trayectoria evolutiva de la tecnología desde sus manifestaciones mecánicas primitivas hasta la actual Cuarta Revolución Industrial (4RI), con el propósito de identificar las condiciones socio-técnicas que dieron origen a la ingeniería de software como una disciplina formal. Se examina la transición crítica del hardware basado en tubos de vacío a la tecnología de estado sólido (transistor), y la evolución de las arquitecturas de procesamiento centralizado hacia sistemas globales interconectados (ARPANET). El análisis se extiende hasta la contemporaneidad, vinculando la soberanía tecnológica y el control de la cadena de suministro de semiconductores (TSMC/ASML) con la capacidad de desarrollar software de alta complejidad. Se concluye que el software no es una entidad independiente, sino una capa de abstracción ontológica condicionada por las fronteras físicas del hardware y las tensiones geopolíticas que definen el acceso a la potencia de cómputo.

Índice

<u>Resumen.....</u>	<u>2</u>
<u>Índice.....</u>	<u>3</u>
<u>1. Objetivos de la práctica.....</u>	<u>4</u>
<u>1.1 Objetivos particulares.....</u>	<u>4</u>
<u>2. Introducción teórica.....</u>	<u>5</u>
<u>2.1 De la mecánica a la electrónica: La abstracción de Lovelace.....</u>	<u>5</u>
<u>2.2 El concepto de programa almacenado (Von Neumann).....</u>	<u>5</u>
<u>3. Desarrollo.....</u>	<u>6</u>
<u>3.1 Matriz analítica de hitos (1800 a.C. – 2026).....</u>	<u>6</u>
<u>3.2 Análisis de contrafacticos.....</u>	<u>8</u>
<u>3.3 Síntesis de impacto técnico.....</u>	<u>8</u>
<u>3.3.1 El transistor y la ruptura de la ineficiencia térmica.....</u>	<u>8</u>
<u>3.3.2 ARPANET y la génesis de la ingeniería de sistemas distribuidos.....</u>	<u>9</u>
<u>4. Actividades complementarias.....</u>	<u>10</u>
<u>4.1 Reflexión técnica.....</u>	<u>10</u>
<u>4.2 Reporte de visita virtual.....</u>	<u>10</u>
<u>4.3 Ensayo: El Alan Turing del futuro y la crisis de la soberanía tecnológica.....</u>	<u>10</u>
<u>4.4 Desafío de código: Lógica de tarjetas perforadas y algoritmo de Fibonacci.....</u>	<u>11</u>
<u>5. Conclusiones.....</u>	<u>12</u>
<u>Fuentes de información.....</u>	<u>13</u>

1. Objetivos de la práctica

Analizar la evolución de la tecnología a lo largo de la historia de la humanidad para comprender el contexto socio-técnico que propició el surgimiento de la ingeniería de software como disciplina, identificando los hitos clave, autores, condiciones históricas y el impacto global de cada innovación en la resolución de problemas de complejidad creciente.

1.1 Objetivos particulares

- Identificar y fechar con precisión técnica los momentos clave en la evolución del hardware y las redes de comunicación.
- Evaluar las condiciones económicas y sociales que forzaron la transición hacia la automatización lógica.
- Determinar los actores institucionales y grupos de interés beneficiados por la innovación tecnológica.
- Construir una línea de tiempo descriptiva que relacione la arquitectura física del computador con los paradigmas de programación resultantes.

2. Introducción teórica

Para la ingeniería de software, comprender su historia no es un ejercicio de nostalgia, sino un análisis de las restricciones que definieron sus métodos. Brooks (1995) argumenta que la esencia del software es la complejidad conceptual, pero esta complejidad sólo pudo manifestarse cuando el hardware superó las limitaciones de tamaño y confiabilidad.

2.1 De la mecánica a la electrónica: La abstracción de Lovelace

El surgimiento de la ingeniería de software se remonta conceptualmente a las notas de Ada Lovelace en 1843. Mientras Charles Babbage veía en la Máquina Analítica un dispositivo para calcular tablas logarítmicas, Lovelace teorizó que cualquier información representable simbólicamente podía ser procesada. Esta es la primera distinción entre hardware (los engranajes) y software (las instrucciones lógicas). Sin embargo, la implementación de estas ideas requería una velocidad de conmutación que la mecánica no podía ofrecer. La transición hacia la electrónica mediante relevadores y, posteriormente, tubos de vacío, permitió la ejecución de algoritmos en milisegundos, pero a un costo energético y de mantenimiento prohibitivo.

2.2 El concepto de programa almacenado (Von Neumann)

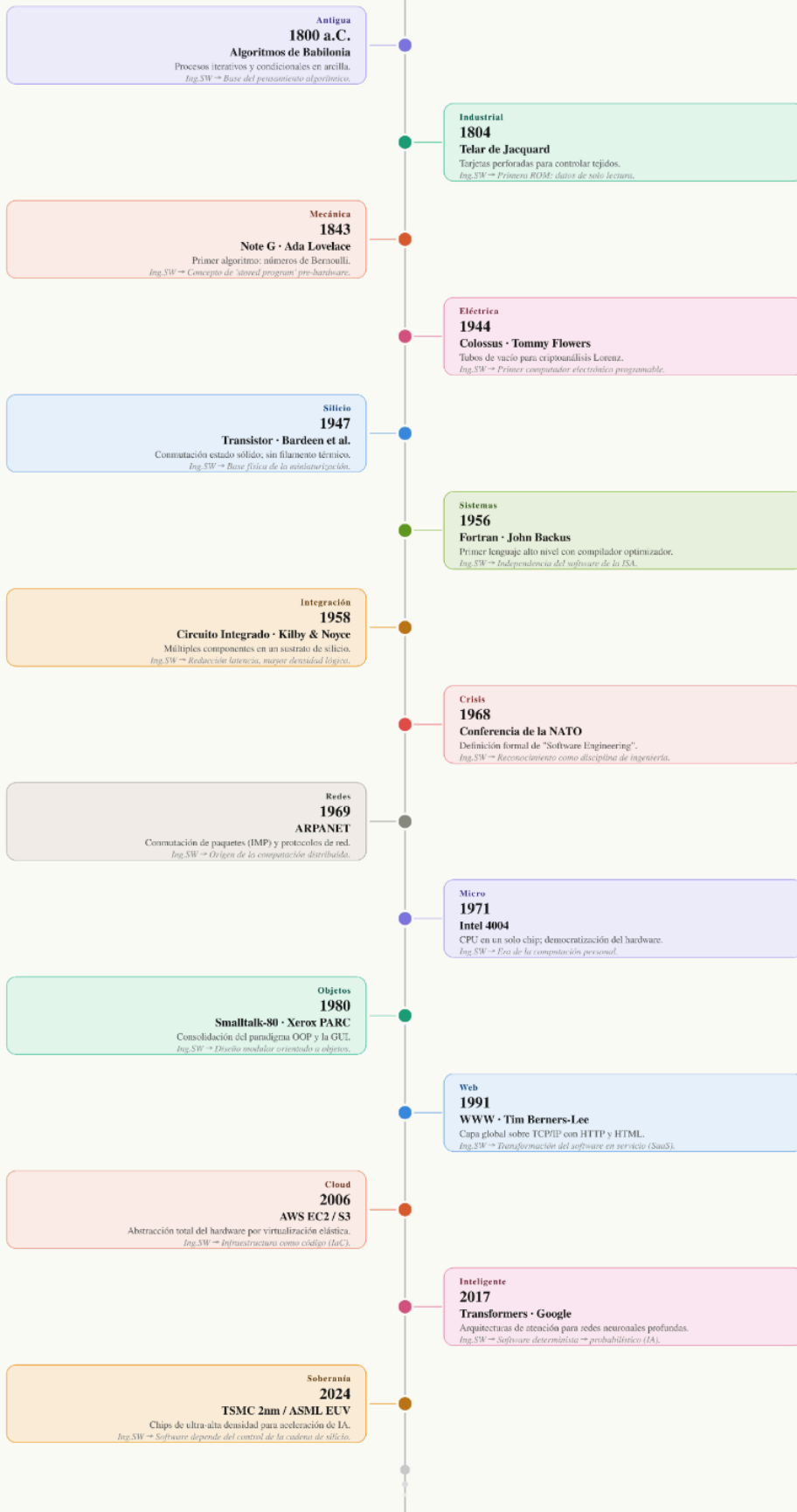
Antes de la arquitectura de Von Neumann, "programar" significaba reconfigurar físicamente el hardware mediante cables y puentes (plugboards), como en el ENIAC. El hito técnico que realmente habilita la ingeniería de software es la noción de que el programa y los datos residen en la misma memoria. Esta unificación permitió que el software se convirtiera en un producto digital manipulable por otro software, dando origen a ensambladores, compiladores y, eventualmente, sistemas operativos complejos (Ceruzzi, 2003).

3. Desarrollo

3.1 Matriz analítica de hitos (1800 a.C. – 2026)

Historia de la Computación

1800 a.C. – 2026 · Matriz analítica de hitos que transformaron la ingeniería de software



Legenda de eras

- Antigua / Micro
- Industrial / Objetos
- Mecánica / Cloud
- Eléctrica / Inteligente
- Silicio / Web
- Sistemas
- Integración / Soberanía
- Crisis
- Redes

A continuación, se presenta un desglose técnico de los hitos que transformaron la computación de una curiosidad matemática a una industria global.

3.2 Análisis de contrafácticos

Si el transistor no hubiese sustituido al tubo de vacío en 1947, la informática moderna se habría enfrentado a un muro térmico insuperable. Una computadora con la potencia de un smartphone actual basada en tubos de vacío requeriría una planta nuclear dedicada para su alimentación y el tamaño de un estadio de fútbol para su enfriamiento. En este escenario, la ingeniería de software habría permanecido como una disciplina de optimización de recursos extremos, similar a la programación de los años 50, donde cada byte era crítico. No existirían los lenguajes de alto nivel (Python, Java), ya que el "overhead" de abstracción sería impagable para el hardware disponible. La "democratización" del software sería inexistente, limitando el acceso a la computación a superpotencias estatales, profundizando la brecha de desigualdad global a niveles de castas tecnológicas.

3.3 Síntesis de impacto técnico

3.3.1 El transistor y la ruptura de la ineficiencia térmica

El transistor es, sin duda, el mayor hito en la historia de la ingeniería de software. Al eliminar la necesidad de calentar un filamento (como en los tubos de vacío), permitió que los componentes lógicos se colocaran a distancias micrométricas sin fundirse. Esto redujo drásticamente el tiempo de viaje de la señal eléctrica, permitiendo frecuencias de reloj que pasaron de KiloHertz a GigaHertz. Para la ingeniería de software, esto significó la libertad de crear múltiples capas de abstracción. Ya no era necesario escribir en código máquina; el hardware era lo suficientemente rápido para soportar sistemas operativos, máquinas virtuales y recolectores de basura de memoria (Garbage Collectors) sin una degradación perceptible del rendimiento. La robustez y la escala de los sistemas de software actuales son un producto directo de la estabilidad de estado sólido que el silicio proporcionó.

3.3.2 ARPANET y la génesis de la ingeniería de sistemas distribuidos

ARPANET no solo inventó el Internet; inventó la forma en que los ingenieros de software piensan sobre la resiliencia. Al adoptar la conmutación de paquetes en lugar de la conmutación de circuitos, el software pasó de ser un ente estático en una CPU a ser una red de procesos colaborativos que deben manejar la incertidumbre (latencia, pérdida de paquetes). La ingeniería de software moderna (microservicios, APIs, Cloud Computing) es la hija directa de los protocolos TCP/IP desarrollados para ARPANET. Sin esta red, no habría soberanía de datos ni colaboración masiva en proyectos Open Source, y el software seguiría limitado por la capacidad física de almacenamiento local de una sola máquina.

4. Actividades complementarias

4.1 Reflexión técnica

En el análisis del documental sobre Ada Lovelace, se destaca su capacidad de "Ciencia Poética". Sin embargo, desde una perspectiva técnica de posgrado, su mayor contribución fue el reconocimiento de que la estructura de control (el flujo del programa) es independiente de la naturaleza de los datos procesados. En su "Note G", al detallar el cálculo de los números de Bernoulli, Lovelace introduce conceptos de bucles anidados y subrutinas. Lovelace predijo que si se podía cuantificar la armonía musical, la máquina podría componer música. Esta es la primera definición de una máquina de Turing universal, un siglo antes de Turing. Su visión abstracta es la que hoy permite que el mismo ingeniero de software que diseña un sistema financiero pueda aplicar sus conocimientos al diagnóstico médico por IA, ya que la lógica subyacente es universal.

4.2 Reporte de visita virtual

Durante el recorrido por los archivos del CHM, se identifican piezas clave que dictaron las convenciones del software actual:

- **IBM System/360:** Fue la primera arquitectura que separó la implementación del hardware de la interfaz del software. Antes del 360, el software de una máquina no corría en otra. El S/360 forzó la creación de estándares de compatibilidad, permitiendo que la industria del software se convirtiera en un mercado independiente del hardware.
- **Cray-1:** La arquitectura circular del Cray-1 muestra el límite físico de la velocidad de la luz. Los cables cortos eran necesarios para que las señales llegaran a tiempo a los registros vectoriales. Esto enseña a los ingenieros de software que, en sistemas de alto rendimiento, el diseño del software debe estar íntimamente ligado a la topología física de la memoria y el bus.
- **Xerox Alto:** En este dispositivo se observa el nacimiento de la Programación Orientada a Objetos y la Interfaz Gráfica de Usuario (GUI). El software dejó de ser texto secuencial para convertirse en un ecosistema de objetos interactivos.

4.3 Ensayo: El Alan Turing del futuro y la crisis de la soberanía tecnológica

¿Quién será el Alan Turing del futuro? En un mundo donde el software está alcanzando la frontera de la Inteligencia Artificial General (AGI), el nuevo Turing no será un experto en

lógica pura, sino un arquitecto de la "Soberanía del Silicio". El problema que deberá resolver es el de la **Alineación y la Escasez Física**. Como se analizó en la presentación de la 4RI, el control de la tecnología hoy reside en un cuello de botella físico: las máquinas EUV de ASML y las plantas de TSMC en Taiwán. El Turing del futuro será aquel que logre desarrollar algoritmos de IA que no requieran infraestructuras energéticas masivas ni hardware de 2 nanómetros inaccesible para la mayoría de las naciones. El desafío actual es que el software se ha vuelto tan pesado que solo las Big Tech pueden "entrenarlo". El Alan Turing del futuro resolverá el problema de la **Democratización Algorítmica**, creando un marco de software que pueda alcanzar la consciencia o la resolución de problemas complejos mediante arquitecturas de computación distribuida o bio-computación, rompiendo la dependencia del silicio geopolíticamente restringido. Este genio deberá descifrar el "código de la consciencia" para asegurar que la tecnología sea un bien público y no una herramienta de hegemonía digital.

4.4 Desafío de código: Lógica de tarjetas perforadas y algoritmo de Fibonacci

Para un ingeniero, la tarjeta perforada representa la primera memoria de estado sólido persistente. Utilizando el estándar Hollerith, podemos simular la generación de la serie de Fibonacci mediante una secuencia de operaciones en registros.

Lógica de Ejecución (Manual de Tarjeta):

1. **Tarjeta 01 (Carga):** Se perforan los valores iniciales (0 en RegA, 1 en RegB).
2. **Tarjeta 02 (Suma):** El sumador mecánico lee las posiciones de RegA y RegB, perforando el resultado en una nueva columna (RegC).
3. **Tarjeta 03 (Swap):** El valor de RegB se desplaza a RegA; el valor de RegC se desplaza a RegB.
4. **Tarjeta 04 (Loop):** Un salto condicional mecánico que reinicia el ciclo si no se ha alcanzado la perforación de "fin de proceso".

Representación binaria en cuadrícula de papel (Primeros 5 términos): Col 01: 00000001 (1) Col 02: 00000001 (1) Col 03: 00000010 (2) Col 04: 00000011 (3) Col 05: 00000101 (5) *Cada bit "1" representa una perforación física que permite el paso de un sensor mecánico.*

5. Conclusiones

La evolución tecnológica demuestra que la Ingeniería de Software es el resultado de un proceso de liberación de las restricciones físicas. Empezamos programando con engranajes y cables, y hoy programamos infraestructuras invisibles en la nube. Sin embargo, la actual crisis de los semiconductores y la disputa por Taiwán nos devuelven a la realidad: el software no existe sin el hardware. Como ingenieros de software en la era de la 4RI, nuestra responsabilidad es triple:

1. Dominar la abstracción lógica para gestionar la complejidad creciente.
2. Comprender la infraestructura física (semiconductores/redes) para optimizar el rendimiento y la eficiencia energética.
3. Participar en la discusión geopolítica sobre la soberanía tecnológica, asegurando que el desarrollo de software en México no sea solo consumo de tecnología extranjera, sino creación de activos originales bajo los modelos de transferencia analizados en prácticas anteriores.

Fuentes de información

- Brooks, F. P. (1995). *The Mythical Man-Month: Essays on Software Engineering* (Anniversary ed.). Addison-Wesley.
- Ceruzzi, P. E. (2003). *A History of Modern Computing*. MIT Press.
- Isaacson, W. (2014). *The Innovators: How a Group of Hackers, Geniuses, and Geeks Created the Digital Revolution*. Simon & Schuster.
- Naughton, J. (2016). *A Brief History of the Future: Origins of the Internet*. Hachette UK.
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
- Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5th ed.). Pearson.
- WIPO (2024). *Global Innovation Index 2024: Unlocking the promise of social innovation*. World Intellectual Property Organization.
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education.
- Sommerville, I. (2011). *Software Engineering* (9th ed.). Pearson.